

Day 1: Practical expectation of both IBM IJP roles

5-line beginner summary

1. **JD 1 is more AI solution architecture focused:** LLMs, NLP, foundation models, Watson/IBM AI stack, and full AI lifecycle.
 2. **JD 2 is more data platform + GenAI engineering focused:** Databricks, Delta Lake, ETL/ELT, MLflow, RAG, vector DB, LangChain, and deployment.
 3. Both roles need **Python, ML/AI thinking, cloud, APIs, data handling, governance, and communication.**
 4. A **Band 08 Data Scientist** is expected to work independently, design solutions, guide others, and explain business impact.
 5. Your backend, cloud, CI/CD, automation, API, and pipeline experience can strongly support both roles if you connect it to **production AI systems.**
-

1. Difference between JD 1 and JD 2

Think of both roles as building AI solutions, but their **center of gravity** is different.

JD 1: Data Scientist - Artificial Intelligence

JD 1 is closer to:

“Can you design and deliver AI/LLM solutions for business problems using IBM/cloud AI technologies?”

Main focus areas:

- Foundation models
- LLMs
- NLP and ML
- Python AI frameworks
- SQL/NoSQL
- Cloud platforms
- Watson / IBM AI technologies
- AI solution architecture
- End-to-end AI lifecycle

This role expects you to understand how AI models solve business problems. You should be able to explain:

- What model to use
- Why that model is suitable
- How data flows into the model
- How output is validated
- How the AI solution is deployed
- How risks like hallucination, bias, privacy, and governance are handled

Simple way to remember:

JD 2: Data Scientist - Advanced Analytics / GenAI / Databricks

JD 2 is closer to:

“Can you build production-grade GenAI and analytics pipelines using Databricks, RAG, MLflow, APIs, and MLOps?”

Main focus areas:

- Databricks
- Delta Lake
- ETL/ELT
- MLflow
- RAG and advanced RAG
- Vector databases
- Reranking
- LangChain, LangGraph, CrewAI, Semantic Kernel
- REST API deployment
- MLOps and governance
- Cloud platforms

This role expects you to understand the **data engineering + GenAI engineering side**. You should be able to explain:

- How raw data becomes clean data
- How data is stored in Delta Lake
- How embeddings are generated
- How vector search works
- How RAG answers are generated
- How MLflow tracks experiments
- How models/APIs are deployed
- How production monitoring and governance are handled

Simple way to remember:

| JD 2 = **Databricks + RAG + MLOps + production GenAI pipelines**

2. Common skills required in both roles

Both roles need these core skills:

Python

You should be comfortable with:

- Functions
- Classes
- Error handling

- Logging
- APIs
- Data processing
- Pandas basics
- ML/AI libraries
- Writing clean reusable code

Interview expectation:

| They may not ask only syntax. They may ask how you structure AI code for production.

Example:

```
data_loader.py
preprocess.py
model_service.py
api.py
config.py
logger.py
```

Machine Learning and AI basics

You should know:

- Classification
- Regression
- Clustering
- NLP basics
- Embeddings
- LLMs
- Prompting
- RAG
- Evaluation
- Bias and hallucination

You do not need to sound like a PhD researcher. But you should explain clearly:

| “Given this business problem, I will choose this AI approach, because...”

Data skills

Both JDs require data understanding:

- SQL
- NoSQL
- Data quality
- Data cleaning
- Metadata
- Structured and unstructured data

- Data pipelines

Example:

Structured data: Customer ID, product ID, order amount Unstructured data: PDFs, emails, support tickets, policy documents
--

Cloud and deployment thinking

You should understand:

- Where code runs
- Where data is stored
- How APIs expose AI services
- How logs and monitoring work
- How CI/CD deploys changes
- How secrets/configuration are managed

This is where your existing backend/cloud/CI-CD experience becomes valuable.

Governance and responsible AI

Both roles expect awareness of:

- Data privacy
- Access control
- Model explainability
- Audit logs
- Bias
- Hallucination
- Human review
- Secure deployment

For IBM roles, governance is important because enterprise clients care about trust, compliance, and repeatability.

3. Skills more important for JD 1

For JD 1, focus more on **AI architecture and model solutioning**.

Important skills:

Foundation models and LLMs

You should understand:

- What LLMs are
- How prompts work

- What tokens are
- What context window means
- What hallucination means
- When to use RAG instead of fine-tuning
- When to use smaller vs larger models

Interview-ready explanation:

“A foundation model is a large pre-trained model that can be adapted for many tasks like summarization, Q&A, classification, and content generation. In enterprise use cases, we often combine it with RAG so the model answers using trusted company data.”

NLP and ML

You should revise:

- Text classification
- Sentiment analysis
- Named entity recognition
- Summarization
- Semantic search
- Embeddings
- Traditional ML vs LLM-based approach

Example:

Business problem:
Classify customer complaints into billing, network, refund, and technical issue.

Traditional ML:
Train classifier using labeled complaints.

LLM approach:
Prompt LLM or use embeddings + classifier.

RAG approach:
Retrieve policy documents and generate answer with citations.

AI solution architecture

You should be able to design this:

User -> API -> Prompt Builder -> LLM/RAG Service -> Response Validator -> Final Answer

They may ask:

- How will you design a chatbot?
- How will you reduce hallucination?
- How will you evaluate answer quality?

- How will you secure user data?
 - How will you monitor model performance?
-

Watson / IBM AI stack awareness

You should be ready to discuss IBM AI tools at a high level:

- Watson / watsonx-style AI platforms
- Model development
- Model governance
- Enterprise AI deployment
- AI lifecycle management

You do not need to memorize every product feature on Day 1. But you should connect IBM's AI focus with:

trusted AI + enterprise governance + scalable AI solutions
--

4. Skills more important for JD 2

For JD 2, focus more on **Databricks, data pipelines, RAG, and MLOps**.

Databricks

You should understand:

- Workspace
- Notebooks
- Clusters
- Jobs
- Delta Lake
- Unity Catalog
- MLflow integration
- Data pipeline orchestration

Beginner explanation:

Databricks is a cloud data and AI platform where teams clean data, build ML models, track experiments, and deploy analytics/AI solutions at scale.
--

Delta Lake

Delta Lake helps with:

- Reliable data storage
- ACID transactions
- Schema enforcement
- Versioning
- Time travel

- Batch and streaming data

Simple example:

Raw customer support tickets come daily.
Delta Lake stores cleaned and versioned data.
If bad data enters, we can trace or roll back.

ETL / ELT

ETL means:

Extract -> Transform -> Load

ELT means:

Extract -> Load -> Transform

For JD 2, you should know how data flows from raw source to analytics/AI-ready tables.

Example:

Raw PDFs / tickets / logs
↓
Clean text
↓
Chunk text
↓
Generate embeddings
↓
Store in vector DB
↓
Use for RAG

MLflow

MLflow is important for:

- Experiment tracking
- Model versioning
- Metrics tracking
- Model registry
- Deployment lifecycle

Interview-ready explanation:

“MLflow helps us track which model version, dataset, parameters, and metrics produced a result. This is important in production because we need reproducibility and governance.”

RAG and advanced RAG

Basic RAG:

User question -> Retrieve relevant documents -> Send context to LLM -> Generate answer

Advanced RAG may include:

- Better chunking
- Metadata filtering
- Hybrid search
- Reranking
- Query rewriting
- Multi-step retrieval
- Evaluation
- Guardrails
- Citation-based answers

For JD 2, this is very important.

Vector databases

You should understand:

- Embeddings convert text into numbers
- Similar meanings have nearby vectors
- Vector DB stores and searches embeddings
- Used in semantic search and RAG

Example:

Question:
"How do I reset my password?"

Vector DB may retrieve:
"Password reset policy"
"Login troubleshooting guide"
"Account recovery steps"

LangChain, LangGraph, CrewAI, Semantic Kernel

At Day 1 level, understand them like this:

LangChain:
Helps connect LLMs with tools, prompts, memory, retrieval, and APIs.

LangGraph:
Helps build stateful multi-step LLM workflows.

CrewAI:

Helps create multi-agent workflows where agents have different roles.

Semantic Kernel:

Microsoft-style framework for connecting AI prompts, tools, and business logic.

You do not need to master all immediately. For interview, you should know:

- Why these tools exist
- When to use them
- Their risks
- How to keep the system maintainable

5. What a Band 08 Data Scientist is expected to do

In practical IJP terms, a Band 08 role usually expects more than individual task execution.

You are expected to act like someone who can:

Own a problem end to end

Not just:

I trained a model.

But:

I understood the business problem, analyzed data, selected the AI approach, built a prototype, evaluated it, deployed it, monitored it, and explained trade-offs.

Convert vague business problems into AI solutions

Example business ask:

Can we reduce customer support workload?

Band 08 thinking:

1. Understand support ticket types.
2. Identify repetitive questions.
3. Build RAG chatbot using policy documents.
4. Add fallback to human agent.
5. Track answer accuracy and deflection rate.
6. Monitor hallucination and user feedback.

Make architecture decisions

You may need to decide:

- RAG vs fine-tuning
- SQL vs NoSQL
- Batch vs real-time processing

- Simple API vs agentic workflow
 - Open-source model vs enterprise model
 - Manual review vs automated response
 - Cloud deployment strategy
 - Monitoring and rollback approach
-

Communicate with multiple stakeholders

You may interact with:

- Business teams
- Data engineers
- ML engineers
- Backend developers
- Cloud/DevOps teams
- Security teams
- Product owners
- Architects

So communication is very important.

Mentor or guide others

Band 08 may be expected to:

- Review designs
 - Help juniors
 - Create reusable patterns
 - Improve team practices
 - Explain technical decisions clearly
-

6. End-to-end AI project lifecycle

An AI project is not only model training. It has many stages.

Stage 1: Business understanding

Ask:

- What problem are we solving?
- Who will use the solution?
- What is success?
- What risk exists?
- What data is available?

Example:

Problem:

Support agents spend too much time answering repeated policy questions.

Goal:
Reduce manual effort by 30% using AI assistant.

Stage 2: Data collection

Collect:

- PDFs
- Databases
- Logs
- Tickets
- Emails
- Knowledge base articles
- User feedback

Stage 3: Data cleaning and preparation

Clean:

- Duplicates
- Missing values
- Bad formatting
- Sensitive information
- Irrelevant text
- Outdated documents

For RAG:

Clean text -> Chunk text -> Add metadata -> Generate embeddings

Stage 4: Model or AI approach selection

Decide:

Classification?
Prediction?
Summarization?
RAG?
Agent?
Fine-tuning?
Traditional ML?

Example:

For policy Q&A:
Use RAG, not fine-tuning, because answers should come from latest policy documents.

Stage 5: Build prototype

Create small working version:

- Use sample data
 - Build retrieval
 - Add prompt
 - Generate response
 - Test manually
 - Check failure cases
-

Stage 6: Evaluate

Measure:

- Accuracy
- Precision/recall
- Response relevance
- Hallucination rate
- Latency
- Cost
- User satisfaction
- Business impact

For RAG:

Did it retrieve the right document? Did the LLM answer from the document? Did it avoid unsupported claims?
--

Stage 7: Deploy

Deploy as:

- REST API
- Batch job
- Chatbot
- Internal tool
- Dashboard
- Workflow automation

Production needs:

- Authentication
- Logging
- Monitoring
- Error handling
- Retry logic
- CI/CD

- Rollback
 - Alerts
-

Stage 8: Monitor and improve

Track:

- Bad answers
- Slow responses
- Model drift
- Data drift
- User feedback
- API failures
- Cost
- Token usage

Then improve:

- Prompts
 - Chunks
 - Retrieval
 - Reranking
 - Data quality
 - Model choice
 - Guardrails
-

7. How to connect your backend, cloud, CI/CD, and automation experience

This is very important for you.

You already have experience around:

- Backend services
- APIs
- Cloud/platform work
- CI/CD pipelines
- Tekton / pipeline thinking
- Slack automation
- CLI development in Go
- Logging and error trace capture
- Deployment automation
- Docker/container troubleshooting

You should not present this as separate from AI. You should connect it to **production AI engineering**.

How to explain it in interview

You can say:

“My current experience is mainly around backend automation, CI/CD pipelines, cloud/platform workflows, and production reliability. For these AI/Data Scientist roles, I can use the same engineering foundation to productionize AI solutions. For example, an LLM or RAG prototype is not enough by itself. It needs APIs, logging, monitoring, CI/CD, error handling, deployment automation, and governance. That is where my existing experience becomes useful.”

Mapping your experience to these roles

Your experience	How it helps JD 1	How it helps JD 2
Backend API knowledge	Deploy AI services as APIs	Deploy RAG/ML models as REST APIs
CI/CD pipelines	Automate AI solution deployment	Automate Databricks/MLflow/model deployment
Cloud experience	Host AI workloads securely	Run scalable data/ML pipelines
Logging/error handling	Monitor LLM failures	Monitor ETL/RAG/model failures
Slack automation	Alerting for AI pipeline failures	MLOps and data pipeline notifications
Docker/container knowledge	Package AI services	Deploy ML/RAG APIs consistently
Go CLI automation	Build internal platform tools	Automate model/data workflows
Shell scripting	Pipeline orchestration	Batch ETL/ELT automation

Strong interview positioning

You can position yourself like this:

I bring a combination of AI learning, backend engineering, cloud automation, and production mindset.

Many AI projects fail because they remain notebooks or prototypes.

My strength is that I understand how to move systems toward production using APIs, CI/CD, logging, automation, and operational reliability.

This is powerful for both JDs.

8. Easy real-world example

Problem

A company has thousands of internal HR policy documents. Employees ask repetitive questions:

How many leaves can I carry forward?
What is the maternity leave policy?
How do I claim travel reimbursement?

JD 1 approach

JD 1 candidate thinks:

This is an enterprise AI assistant problem.
We can use a foundation model with RAG.
We need trusted answers, governance, privacy, and IBM/cloud AI architecture.

Solution:

```
Employee question
↓
Retrieve HR policy documents
↓
Send relevant content to LLM
↓
Generate answer with citations
↓
Apply guardrails
↓
Return final response
```

JD 1 focus:

- AI architecture
- LLM selection
- Governance
- Watson/enterprise AI integration
- Responsible AI
- Business impact

JD 2 approach

JD 2 candidate thinks:

This needs a data pipeline and production RAG system.
We need Databricks, Delta Lake, embeddings, vector DB, MLflow, API deployment, and monitoring.

Solution:

```
HR PDFs
↓
Databricks ETL job
↓
Clean and chunk documents
↓
Store metadata in Delta Lake
```

```

↓
Generate embeddings
↓
Store vectors in vector DB
↓
RAG API retrieves relevant chunks
↓
Reranker improves results
↓
LLM generates answer
↓
MLflow/logging tracks quality

```

JD 2 focus:

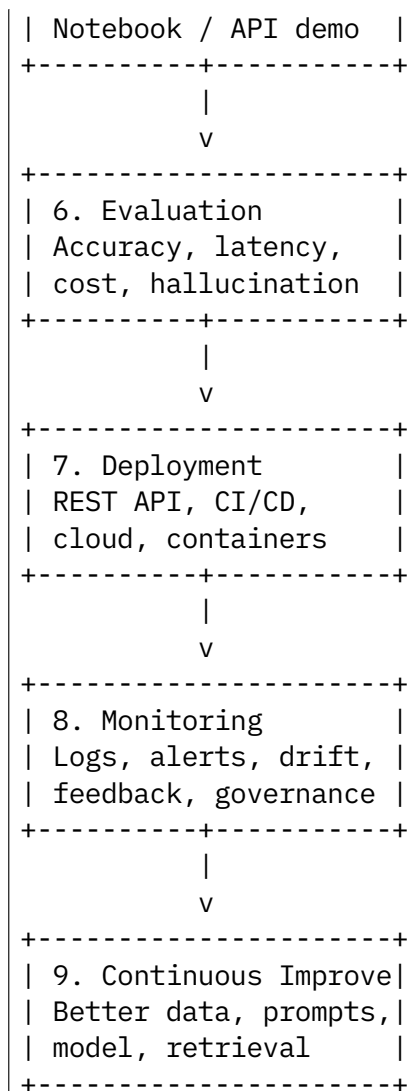
- ETL/ELT
- Delta Lake
- Vector DB
- RAG pipeline
- MLflow
- REST API
- MLOps

ASCII diagram showing AI project lifecycle

```

+-----+
| 1. Business Problem |
| What do users need? |
+-----+
      |
      v
+-----+
| 2. Data Collection  |
| DB, PDFs, logs, APIs |
+-----+
      |
      v
+-----+
| 3. Data Preparation |
| Clean, chunk, label |
+-----+
      |
      v
+-----+
| 4. AI Approach      |
| ML / LLM / RAG      |
+-----+
      |
      v
+-----+
| 5. Prototype        |
+-----+

```

Pseudocode for solving an AI business problem

Business problem:

Reduce customer support workload using an AI assistant.

Pseudocode:

START

DEFINE business_problem = "Reduce repeated customer support questions"

STEP 1: Understand requirement

Talk to business team

Identify common question categories

Define success metrics

Example: reduce manual tickets by 30%

Example: answer accuracy above 85%

STEP 2: Collect data

Get support tickets

Get FAQ documents

- Get policy documents
- Get historical agent responses

STEP 3: Prepare data

- Remove duplicate documents
- Remove sensitive information
- Clean text
- Split long documents into smaller chunks
- Add metadata like document_name, date, category

STEP 4: Choose AI approach

- IF answer must come from latest documents:
 - Choose RAG
- ELSE IF task needs prediction:
 - Choose ML model
- ELSE IF task needs classification:
 - Choose text classifier

STEP 5: Build retrieval system

- Convert each document chunk into embedding
- Store embeddings in vector database
- Store metadata in SQL/Delta table

STEP 6: Build AI response flow

- Accept user question
- Convert question into embedding
- Search similar chunks from vector database
- Rerank retrieved chunks
- Create prompt using question + retrieved context
- Send prompt to LLM
- Generate answer

STEP 7: Validate response

- Check if answer is grounded in retrieved context
- Check if answer contains unsafe content
- Check confidence score
- IF confidence is low:
 - Send to human support
- ELSE:
 - Return answer to user

STEP 8: Deploy

- Wrap solution in REST API
- Add authentication
- Add logging
- Add monitoring
- Deploy using CI/CD pipeline

STEP 9: Monitor

- Track wrong answers
- Track latency
- Track cost
- Track user feedback
- Track failed API calls

STEP 10: Improve
 Improve documents
 Improve chunking
 Improve prompts
 Improve retrieval
 Improve model choice

END

Interview relevance

What interviewers may expect from you

They may check whether you can move from theory to practical delivery.

They may ask:

Tell me about an AI/ML project you worked on.
How would you design a RAG system?
How do you evaluate an LLM application?
How do you reduce hallucination?
How do you deploy a model into production?
How do you monitor model performance?
How do you handle data quality issues?
How do you use MLflow?
What is the difference between RAG and fine-tuning?
How does Databricks help in ML/AI projects?
How will your current experience help in this role?

Strong answer style for you

Use this structure:

1. Business problem
2. Data available
3. AI/ML approach
4. Architecture
5. Deployment
6. Monitoring
7. Business impact

Example:

For an internal document Q&A system, I would first understand the user questions and success metrics. Then I would collect policy documents, clean and chunk them, generate embeddings, and store them in a vector database. For every user question, I would retrieve relevant chunks, optionally rerank them, and send the best context to an LLM. I would expose this through a REST API, deploy it using CI/CD, add logging and monitoring, and track answer quality, latency, and hallucination risk.

Common mistakes

Mistake 1: Saying only “I know LLM”

Better:

I understand how to use LLMs inside an enterprise AI system with data, retrieval, APIs, evaluation, monitoring, and governance.

Mistake 2: Ignoring data pipelines

AI systems depend heavily on data. For JD 2 especially, do not ignore:

- ETL/ELT
- Delta Lake
- data quality
- metadata
- embeddings
- MLflow
- monitoring

Mistake 3: Confusing RAG and fine-tuning

Simple difference:

RAG:

Use external documents at runtime.

Fine-tuning:

Change model behavior by training on examples.

For enterprise Q&A, RAG is often safer because answers can come from updated trusted documents.

Mistake 4: Talking only about notebooks

A notebook is good for experimentation, but interviewers want production thinking:

Notebook -> reusable code -> API -> CI/CD -> deployment -> monitoring

Mistake 5: Not connecting your current experience

Your backend/cloud/automation experience is highly relevant. Do not say:

I do not have direct AI production experience.

Say:

I have strong production engineering experience, and I am applying that to AI systems. I understand that real AI solutions need APIs, cloud deployment, CI/CD, logging, monitoring, and automation.

Mistake 6: Ignoring governance

For IBM roles, always mention:

- data privacy
- access control
- auditability
- explainability
- responsible AI
- human review for risky cases

Final Day 1 mental model

Use this simple memory:

JD 1 = AI Architect Mindset

Focus: LLMs, NLP, foundation models, Watson, AI lifecycle, enterprise solutioning.

JD 2 = GenAI Data Platform Mindset

Focus: Databricks, Delta Lake, ETL, MLflow, RAG, vector DB, MLOps, APIs.

Common Core = Python + ML/AI + Cloud + Data + APIs + Governance + Communication.

Your Advantage = Backend + CI/CD + Cloud + Automation + Production Mindset.

For interviews, your best positioning is:

I can understand the business problem, design an AI solution, build a practical prototype, and help move it toward production using backend, cloud, CI/CD, monitoring, and governance practices.